

С чего начать

Каждая лабораторная находится в отдельной папке `labs/labNN` и выполняется независимо от остальных. Работайте с уже выданными материалами. Для следующей работы используется новая заготовка: переносить в неё незавершённый код предыдущей лабораторной не требуется.

Первый запуск

Настройте Python-окружение по инструкции в `README.md` и установите `pytest`. Затем из корня проекта выполните:

```
python check.py lab01
```

Откройте `labs/lab01/TASK.md`: в нём указаны цель работы, обязательные изменения и порядок выполнения. Сначала разберите задание, затем переходите к исходникам и тестам.

В ЛР1 и ЛР4 тесты проходят ещё до начала работы. Задача — провести указанный в задании рефакторинг, сохранив поведение программы. В остальных лабораторных часть проверок изначально не проходит; их список находится в `STARTER-STATE.md` внутри папки работы.

Если возникла ошибка установки зависимостей или импорта, сообщите преподавателю. Не отключайте тесты, чтобы обойти проблему.

Что читать в коде

Пути ниже указаны относительно папки выбранной лабораторной. Какие файлы разрешено менять, смотрите в `TASK.md`.

Путь	Назначение
<code>app/domain/</code>	Классы предметной области: данные объектов и действия с ними.
<code>app/services/</code>	Код, который организует взаимодействие объектов.
<code>app/services/assembly.py</code>	Создание объектов и передача зависимостей. Этот файл можно менять начиная с ЛР5.
<code>app/api.py</code>	Готовые функции для обращения к сервису.
<code>app/support/</code>	Готовые вспомогательные типы и проверки.

В исходниках встретятся следующие обозначения. Они понадобятся постепенно, по мере выполнения работ.

Обозначение	Что означает
<code>self</code>	Ссылка на экземпляр объекта, с которым работает метод.
<code>@property</code>	Позволяет обращаться к методу чтения как к обычному полю. В ранних работах такие свойства уже подготовлены.
<code>Money</code>	Объект с суммой и валютой. Для денежных значений используется <code>Decimal</code> ; не заменяйте его на <code>float</code> .
<code>ABC, @abstractmethod</code>	Базовый класс для абстракций и отметка метода, который должен реализовать наследник. Используются с ЛР3.
<code>dataclass(frozen=True)</code>	Создаёт класс с ограничением изменения полей экземпляра. Разбирать внутреннее устройство декоратора не требуется.
<code>DomainError</code>	Исключение, которое содержит код ошибки.

Завершение работы

Выполните обязательные изменения и запустите проверки текущей лабораторной. Объём кода сам по себе не оценивается: добавлять классы и дополнительные слои только ради количества не нужно.

На защите один участник может показать успешный вызов, а другой — разобрать отказ или объяснить, как добавить новое поведение. При этом каждый должен понимать решение целиком: преподаватель может спросить любого участника о любом его участке. Отдельный отчёт и собственные тесты не требуются.

Дополнительное задание находится в конце `TASK.md`. Проверка: `python check.py lab01 --bonus`.